

# 数据结构与算法分析

---

Violet

2026-04-10



# Chapter 0

## 目 录

|       |                 |    |
|-------|-----------------|----|
| I     | 数据结构基础          |    |
| 1.    | 绪论 ·····        | 6  |
| 2.    | 线性表 ·····       | 7  |
| 3.    | 栈与队列 ·····      | 8  |
| 4.    | 串与数组 ·····      | 9  |
| II    | 树形结构与图结构        |    |
| 5.    | 树与二叉树基础 ·····   | 11 |
| 6.    | 树的应用 ·····      | 12 |
| 7.    | 图的基本概念与存储 ····· | 13 |
| 8.    | 图的遍历与应用 ·····   | 14 |
| III   | 查找与排序           |    |
| 9.    | 查找技术 ·····      | 16 |
| 10.   | 树形查找结构 ·····    | 17 |
| 11.   | 哈希表 ·····       | 18 |
| 12.   | 内部排序算法 ·····    | 19 |
| 13.   | 高级排序与外部排序 ····· | 20 |
| IV    | 算法设计与分析         |    |
| 14.   | 递归、枚举与模拟 ·····  | 22 |
| 15.   | 分治策略 ·····      | 23 |
| 16.   | 贪心算法 ·····      | 24 |
| 17.   | 动态规划 ·····      | 25 |
| 17.1. | 基本概念 ·····      | 25 |
| 17.2. | 线性 DP ·····     | 25 |
| 17.3. | 背包问题 ·····      | 29 |
| 17.4. | 区间 DP ·····     | 31 |
| 17.5. | 树形 DP ·····     | 33 |

17.6. 状态压缩 DP ..... 35

17.7. 数位 DP ..... 37

17.8. DP 优化技巧总结 ..... 39

17.9. 线性 DP 补充 ..... 40

17.10 概率 DP 与期望 DP ..... 42

17.11 DP 建模思路总结 ..... 43

17.12 DP 问题分类速查 ..... 45

18. 回溯法与分支限界 ..... 47

19. 搜索优化与高级技巧 ..... 48

20. 其他算法技巧 ..... 49

|     |                    |
|-----|--------------------|
| V   | 高级数据结构             |
| 21. | 线段树与树状数组 ..... 51  |
| 22. | 平衡树进阶 ..... 52     |
| 23. | 分块与莫队算法 ..... 53   |
| 24. | 并查集与高级应用 ..... 54  |
| 25. | 字符串高级数据结构 ..... 55 |
| 26. | 图论高级算法 ..... 56    |
| 27. | 网络流与匹配 ..... 57    |
| 28. | 其他高级数据结构 ..... 58  |



# 数据结构基础

|               |   |
|---------------|---|
| 1. 绪论 ·····   | 6 |
| 2. 线性表 ·····  | 7 |
| 3. 栈与队列 ····· | 8 |
| 4. 串与数组 ····· | 9 |

# Chapter 1

## 绪论

---

# Chapter 2

## 线性表

---

# Chapter 3

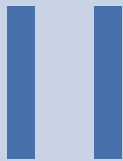
## 栈与队列

---

# Chapter 4

## 串与数组

---



# 树形结构与图结构

|    |           |    |
|----|-----------|----|
| 5. | 树与二叉树基础   | 11 |
| 6. | 树的应用      | 12 |
| 7. | 图的基本概念与存储 | 13 |
| 8. | 图的遍历与应用   | 14 |

# Chapter 5

## 树与二叉树基础

---

# Chapter 6

## 树的应用

---

# Chapter 7

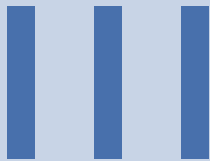
## 图的基本概念与存储

---

# Chapter 8

## 图的遍历与应用

---



# 查找与排序

|                     |    |
|---------------------|----|
| 9. 查找技术 .....       | 16 |
| 10. 树形查找结构 .....    | 17 |
| 11. 哈希表 .....       | 18 |
| 12. 内部排序算法 .....    | 19 |
| 13. 高级排序与外部排序 ..... | 20 |

# Chapter 9

## 查找技术

---

# Chapter 10

## 树形查找结构

---

# Chapter 11

## 哈希表

---

# Chapter 12

## 内部排序算法

---

# Chapter 13

## 高级排序与外部排序

---

# IV

## 算法设计与分析

|                     |    |
|---------------------|----|
| 14. 递归、枚举与模拟 ·····  | 22 |
| 15. 分治策略 ·····      | 23 |
| 16. 贪心算法 ·····      | 24 |
| 17. 动态规划 ·····      | 25 |
| 18. 回溯法与分支限界 ·····  | 47 |
| 19. 搜索优化与高级技巧 ····· | 48 |
| 20. 其他算法技巧 ·····    | 49 |

# Chapter 14

## 递归、枚举与模拟

---

# Chapter 15

## 分治策略

---

# Chapter 16

## 贪心算法

---

# Chapter 17

## 动态规划

动态规划 (Dynamic Programming, 简称 DP) 是一种用于解决复杂优化问题的算法思想, 它通过将问题分解为相互重叠的子问题, 并存储子问题的解 (避免重复计算) 来高效解决问题。

### NOTE

动态规划基于两个重要概念: **最优子结构**和**重叠子问题**。

## 1 基本概念

### 1.1 核心思想

动态规划的核心在于以下两个关键性质:

- **最优子结构**: 一个问题的最优解包含其子问题的最优解。这意味着我们可以通过组合子问题的最优解来构造原问题的最优解。
- **重叠子问题**: 在递归求解过程中, 相同的子问题会被多次计算。动态规划通过存储这些子问题的解 (记忆化), 避免了重复计算。

### 1.2 基本步骤

解决动态规划问题通常遵循以下步骤:

1. **定义状态**: 找出问题中的变量, 用状态表示子问题的解
2. **确定状态转移方程**: 建立不同状态之间的关系 (递推公式)
3. **设置初始条件**: 确定最简单子问题的解 (边界条件)
4. **确定计算顺序**: 自底向上或自顶向下地计算所有状态
5. **构造最优解**: 通过记录的信息重建最优解

### TIP

判断一个问题是否适合用动态规划解决: 看它是否具有最优子结构和重叠子问题两个特征。

## 2 线性 DP

线性 DP 是动态规划中最基础的一类, 具有以下特点:

- 状态转移只依赖前面的有限状态 (通常为前一个状态)
- 问题状态可以表示为一维数组
- 时间复杂度通常为  $O(n)$  的问题

### 2.1 经典问题

## I 最长递增子序列 (LIS)

最长递增子序列 (Longest Increasing Subsequence) 问题是指在一个给定的整数序列 `nums` 中, 找到最长的严格递增子序列的长度。

状态定义: 设 `dp[i]` 表示以 `nums[i]` 结尾的最长递增子序列的长度。

状态转移方程: 对于每个 `i`, 遍历 `j` 从 `0` 到 `i-1`: 如果 `nums[j] < nums[i]`, 则更新 `dp[i]`:  $dp[i] = \max(dp[i], dp[j] + 1)$

初始条件: 每个元素自身可以作为长度为 1 的递增子序列, 因此 `dp[i] = 1` 对所有 `i`。

```
1  int lengthOfLIS(vector<int>& nums) {  
2      if (nums.empty()) return 0;  
3      int n = nums.size();  
4      vector<int> dp(n, 1); // 初始化 dp 数组, 长度为 1  
5  
6      for (int i = 1; i < n; ++i) {  
7          for (int j = 0; j < i; ++j) {  
8              if (nums[j] < nums[i]) {  
9                  dp[i] = max(dp[i], dp[j] + 1);  
10             }  
11         }  
12     }  
13  
14     return *max_element(dp.begin(), dp.end()); // 返回最长递增子序列的长度  
15 }
```

### NOTE

该算法的时间复杂度为  $O(n^2)$ 。可以使用二分查找优化到  $O(n \log n)$ 。

## I 最大子数组和 (Kadane's Algorithm)

最大子数组和 (Maximum Subarray Sum) 问题是指在一个给定的整数数组 `nums` 中, 找到一个连续子数组 (至少包含一个元素), 使得该子数组的和最大。

状态定义: 设 `dp[i]` 表示以 `nums[i]` 结尾的最大子数组和。

状态转移方程:  $dp[i] = \max(nums[i], dp[i-1] + nums[i])$ , 即要么选择当前元素作为新的子数组开始, 要么将当前元素加入到之前的子数组中。

初始条件:  $dp[0] = nums[0]$ 。

```
1  int maxSubArray(vector<int>& nums) {  
2      if (nums.empty()) return 0;  
3      int n = nums.size();  
4      vector<int> dp(n);  
5      dp[0] = nums[0];  
6      int maxSum = dp[0];  
7  
8      for (int i = 1; i < n; ++i) {  
9          dp[i] = max(nums[i], dp[i-1] + nums[i]);  
10         maxSum = max(maxSum, dp[i]);  
11     }
```

```
12
13     return maxSum; // 返回最大子数组和
14 }
```

**TIP**

Kadane 算法可以进一步优化空间复杂度到  $O(1)$ ，只需要维护一个变量即可。

## 打家劫舍 (House Robber)

打家劫舍 (House Robber) 问题是指在一个给定的整数数组 `nums` 中，表示每个房屋中的金额，要求在不触动相邻房屋警报器的情况下，计算能够抢劫到的最高金额。

状态定义：设 `dp[i]` 表示抢劫到第 `i` 个房屋时能够获得最高金额。

状态转移方程： $dp[i] = \max(dp[i-1], dp[i-2] + nums[i])$ ，即要么不抢第 `i` 个房屋，金额为 `dp[i-1]`，要么抢第 `i` 个房屋，金额为 `dp[i-2] + nums[i]`。

初始条件： $dp[0] = nums[0]$ ， $dp[1] = \max(nums[0], nums[1])$ 。

```
1  int rob(vector<int>& nums) {
2      if (nums.empty()) return 0;
3      int n = nums.size();
4      if (n == 1) return nums[0];
5
6      vector<int> dp(n);
7      dp[0] = nums[0];
8      dp[1] = max(nums[0], nums[1]);
9
10     for (int i = 2; i < n; ++i) {
11         dp[i] = max(dp[i-1], dp[i-2] + nums[i]);
12     }
13
14     return dp[n-1]; // 返回最高金额
15 }
```

## 斐波那契数列

计算第  $n$  个斐波那契数： $F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2)$

自顶向下 (记忆化递归)

```
1  // 记忆化数组
2  private static Map<Integer, Integer> memo = new HashMap<>();
3
4  public static int fibonacci(int n) {
5      // 边界条件
6      if (n <= 1) {
7          return n;
8      }
9
10     // 如果已经计算过，则直接返回
11     if (memo.containsKey(n)) {
12         return memo.get(n);
13     }
```

```
13     }
14
15     // 计算并存储结果
16     int result = fibonacci(n - 1) + fibonacci(n - 2);
17     memo.put(n, result);
18     return result;
19 }
```

- 时间复杂度:  $O(n)$  (因为每个值只计算一次)
- 空间复杂度:  $O(n)$  (用于存储递归调用栈和记忆化数组)

### 自底向上 (迭代)

```
1 public static int fibonacci(int n) { Java
2     // 边界条件
3     if (n <= 1) {
4         return n;
5     }
6
7     // 定义数组存储计算结果
8     int[] dp = new int[n + 1];
9     dp[0] = 0;
10    dp[1] = 1;
11
12    // 从第 2 个数开始迭代计算
13    for (int i = 2; i <= n; i++) {
14        dp[i] = dp[i - 1] + dp[i - 2];
15    }
16
17    // 返回第 n 个 Fibonacci 数
18    return dp[n];
19 }
```

- 时间复杂度:  $O(n)$  (线性计算)
- 空间复杂度:  $O(n)$  (用于存储动态规划数组)

### 空间优化

```
1 public static int fibonacci(int n) { Java
2     // 边界条件
3     if (n <= 1) {
4         return n;
5     }
6
7     // 只保存最近两个数
8     int prev1 = 0; // F(n-2)
9     int prev2 = 1; // F(n-1)
10
11    // 迭代计算
12    for (int i = 2; i <= n; i++) {
13        int current = prev1 + prev2; // 当前 Fibonacci 数
14        prev1 = prev2; // 更新 F(n-2)
15        prev2 = current; // 更新 F(n-1)
16    }
```

```
17
18     // 返回第 n 个 Fibonacci 数
19     return prev2;
20 }
```

- 时间复杂度:  $O(n)$  (线性计算)
- 空间复杂度:  $O(1)$  (只存储两个数)

**NOTE**

斐波那契数列是理解动态规划的经典入门案例, 展示了从递归到记忆化再到迭代的优化过程。

## 3 背包问题

背包问题是动态规划中的经典问题类型, 具有重要的理论和实践价值。

### 3.1 0-1 背包问题

给定一个容量为  $W$  的背包和  $n$  件物品, 每件物品有重量  $wt[i]$  和价值  $val[i]$ , 我们需要选择一些物品装入背包, 使得总价值最大化, 同时总重量不超过  $W$ 。

#### 状态分析

状态是动态规划中用来描述子问题的变量。分析状态需要明确:

- 问题的规模: 选择前  $i$  个物品时, 当前的背包容量为  $w$
- 状态的定义: 用  $dp[i][w]$  表示前  $i$  件物品在背包容量  $w$  下的最大价值
- 状态的范围:
  - $i$  的范围是从 0 到  $n$
  - $w$  的范围是从 0 到  $W$

#### 状态转移方程

状态转移方程用于描述如何从一个状态转移到另一个状态。它的确定过程如下:

决策分析: 对于第  $i$  件物品, 我们有两种决策:

1. 不选择第  $i$  件物品: 当前的最大价值是不放第  $i$  件物品的情况, 即:  $dp[i][w] = dp[i-1][w]$
2. 选择第  $i$  件物品: 当前的最大价值是放第  $i$  件物品的情况。此时剩余背包容量为  $w-wt[i-1]$ , 背包中的价值增加了  $val[i-1]$ , 即:  $dp[i][w] = dp[i-1][w-wt[i-1]] + val[i-1]$

选择最大值: 在两种决策中, 我们选择价值更大的决策:

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-wt[i-1]] + val[i-1])$$

边界条件:

- 如果没有物品 (即  $i = 0$ ), 最大价值为 0, 即:  $dp[0][w] = 0 \quad \forall w$
- 如果背包容量为 0 (即  $w = 0$ ), 最大价值也为 0, 即:  $dp[i][0] = 0 \quad \forall i$

#### 二维数组实现

```
1 public static int knapsack(int[] wt, int[] val, int W) {
```

 Java

```

2     int n = wt.length;
3     int[][] dp = new int[n + 1][W + 1];
4
5     for (int i = 1; i <= n; i++) {
6         for (int w = 1; w <= W; w++) {
7             if (wt[i - 1] <= w) {
8                 dp[i][w] = Math.max(dp[i - 1][w], dp[i - 1][w - wt[i - 1]] + val[i - 1]);
9             } else {
10                dp[i][w] = dp[i - 1][w];
11            }
12        }
13    }
14
15    return dp[n][W];
16 }

```

## 一维数组优化（空间优化）

在常规的二维数组 `dp[i][w]` 中，当前状态只依赖上一行的状态，因此可以优化为一维数组

`dp[w]`：从容量  $W$  到  $0$  逆序更新，避免覆盖上一行的数据。

```

1 public static int knapsackOptimized(int[] wt, int[] val, int W) {
2     int n = wt.length;
3     int[] dp = new int[W + 1];
4
5     for (int i = 0; i < n; i++) {
6         for (int w = W; w >= wt[i]; w--) {
7             dp[w] = Math.max(dp[w], dp[w - wt[i]] + val[i]);
8         }
9     }
10
11    return dp[W];
12 }

```

### TIP

0-1 背包的一维优化关键在于逆序遍历容量，这样可以保证每个物品只被选择一次。

## 完全背包与多重背包

**完全背包**：每种物品有无限个可用

- 状态转移方程类似，但遍历容量时需要正序遍历
- 代码只需将内层循环改为：`for (int w = wt[i]; w <= W; w++)`

**多重背包**：每种物品有有限个（数量为 `count[i]`）

- 可以将多重背包转化为 0-1 背包（二进制优化）
- 或者使用单调队列优化到  $O(nW)$

### CAUTION

完全背包和 0-1 背包的唯一区别在于遍历顺序：完全背包正序，0-1 背包逆序。这是一个容易混淆的点。

## 4 区间 DP

区间 DP 是动态规划的一种重要类型，专门解决涉及区间操作的问题。

### 4.1 核心思想

区间 DP 的核心思想包括：

- 状态定义： $dp[i][j]$  表示区间  $[i, j]$  上的最优解
- 区间划分：将大区间划分为两个或多个小区间求解
- 转移策略：通过枚举分割点  $k$  将区间分为  $[i, k]$  和  $[k + 1, j]$
- 合并子解：将小区间的最优解合并得到大区间的解

### 4.2 经典问题

#### 石子合并问题

有  $N$  堆石子排成一排，每堆石子有一定的数量。现要将  $N$  堆石子并成一堆。合并的过程只能每次将相邻的两堆石子堆成一堆，每次合并花费的代价为这两堆石子的和，经过  $N-1$  次合并后成为一堆。求出总的代价最小值。

状态定义： $dp[i][j]$  表示合并第  $i$  堆到第  $j$  堆石子的最小代价

状态转移方程： $dp[i][j] = \min(dp[i][k] + dp[k + 1][j] + sum[i][j])$ ，其中  $i \leq k < j$ ， $sum[i][j]$  表示第  $i$  堆到第  $j$  堆石子的总和

初始条件： $dp[i][i] = 0$ （单堆石子不需要合并）

计算顺序：按区间长度从小到大计算

```
1  int mergeStones(vector<int>& stones) {  
2      int n = stones.size();  
3      vector<vector<int>> dp(n, vector<int>(n, INT_MAX));  
4      vector<int> prefixSum(n + 1, 0);  
5  
6      // 计算前缀和  
7      for (int i = 0; i < n; ++i) {  
8          prefixSum[i + 1] = prefixSum[i] + stones[i];  
9      }  
10  
11     // 初始化：单堆石子代价为0  
12     for (int i = 0; i < n; ++i) {  
13         dp[i][i] = 0;  
14     }  
15  
16     // 按区间长度从小到大计算  
17     for (int len = 2; len <= n; ++len) {  
18         for (int i = 0; i <= n - len; ++i) {  
19             int j = i + len - 1;  
20             int sum = prefixSum[j + 1] - prefixSum[i];  
21  
22             for (int k = i; k < j; ++k) {
```

```

23         dp[i][j] = min(dp[i][j], dp[i][k] + dp[k + 1][j] + sum);
24     }
25 }
26 }
27
28 return dp[0][n - 1];
29 }

```

**NOTE**

石子合并问题的时间复杂度为  $O(n^3)$ ，可以通过四边形不等式优化到  $O(n^2)$ 。

## I 最长回文子序列

给定一个字符串  $s$ ，找到其中最长的回文子序列的长度。

状态定义： $dp[i][j]$  表示字符串  $s$  中从位置  $i$  到位置  $j$  的最长回文子序列长度

状态转移方程：

- 如果  $s[i] == s[j]$ :  $dp[i][j] = dp[i + 1][j - 1] + 2$
- 如果  $s[i] \neq s[j]$ :  $dp[i][j] = \max(dp[i + 1][j], dp[i][j - 1])$

初始条件： $dp[i][i] = 1$  (单个字符是回文)

```

1  int longestPalindromeSubseq(string s) {
2      int n = s.size();
3      vector<vector<int>> dp(n, vector<int>(n, 0));
4
5      // 单个字符的回文长度为1
6      for (int i = 0; i < n; ++i) {
7          dp[i][i] = 1;
8      }
9
10     // 按区间长度从小到大计算
11     for (int len = 2; len <= n; ++len) {
12         for (int i = 0; i <= n - len; ++i) {
13             int j = i + len - 1;
14             if (s[i] == s[j]) {
15                 dp[i][j] = dp[i + 1][j - 1] + 2;
16             } else {
17                 dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
18             }
19         }
20     }
21
22     return dp[0][n - 1];
23 }

```

## 4.3 优化技巧

### I 四边形不等式优化

对于某些区间 DP 问题，如果满足四边形不等式条件，可以将时间复杂度从  $O(n^3)$  优化到  $O(n^2)$ 。

**四边形不等式：**如果函数  $w(i, j)$  满足：

$$w(a, c) + w(b, d) \leq w(a, d) + w(b, c) \quad (a \leq b \leq c \leq d)$$

则可以使用四边形不等式优化。

**TIP**

四边形不等式优化的关键是证明决策单调性，即最优分割点随区间右端点增加而单调不减。

## 环状结构处理（破坏成链）

对于环形结构的区间 DP 问题，常用的技巧是**破坏成链**：

- 将原数组复制一份接到末尾，形成长度为  $2n$  的数组
- 在这个扩展的数组上进行区间 DP
- 最终答案在所有长度为  $n$  的区间中取最优值

## 5 树形 DP

树形动态规划是一种在树结构上进行的动态规划算法，它利用树的递归特性，通过深度优先遍历（DFS）自底向上或自顶向下地求解问题。

### 5.1 特点

树形 DP 具有以下特点：

- **树形结构：**问题基于树或森林结构（无向无环图）
- **递归性质：**问题可以分解为子树上的子问题
- **最优子结构：**整体最优解包含子树最优解
- **无后效性：**当前决策只影响子树，不影响祖先节点

### 5.2 经典问题

#### 树的直径

树的直径是指树上任意两点间的最长路径。

**解题思路：**

- 对每个节点，计算经过该节点的最长路径
- 最长路径 = 该节点到其子树中最深节点的距离 + 次深节点的距离
- 树的直径是所有节点中最长路径的最大值

```
1  int diameter = 0;
2
3  int dfs(TreeNode* node) {
4      if (!node) return 0;
5
6      int leftDepth = dfs(node->left);
7      int rightDepth = dfs(node->right);
8
9      // 更新直径
10     diameter = max(diameter, leftDepth + rightDepth);
```



```
11
12 // 返回当前节点的深度
13 return max(leftDepth, rightDepth) + 1;
14 }
15
16 int diameterOfBinaryTree(TreeNode* root) {
17     dfs(root);
18     return diameter;
19 }
```

## 没有上司的舞会（最大独立集）

在一棵树中，选择一些节点使得任意两个被选节点不相邻，求选中节点权值和的最大值。

状态定义：

- $dp[u][0]$ ：不选择节点  $u$  时，以  $u$  为根的子树的最大权值和
- $dp[u][1]$ ：选择节点  $u$  时，以  $u$  为根的子树的最大权值和

状态转移方程：

- $dp[u][0] = \sum(\max(dp[v][0], dp[v][1]))$ ，其中  $v$  是  $u$  的子节点
- $dp[u][1] = weight[u] + \sum(dp[v][0])$

最终答案： $\max(dp[root][0], dp[root][1])$

```
1  vector<vector<int>> adj;
2  vector<int> weight;
3  vector<vector<int>> dp;
4
5  void dfs(int u, int parent) {
6      dp[u][0] = 0;
7      dp[u][1] = weight[u];
8
9      for (int v : adj[u]) {
10         if (v == parent) continue;
11         dfs(v, u);
12
13         dp[u][0] += max(dp[v][0], dp[v][1]);
14         dp[u][1] += dp[v][0];
15     }
16 }
17
18 int maxIndependentSet(int n) {
19     dp.resize(n, vector<int>(2, 0));
20     dfs(0, -1);
21     return max(dp[0][0], dp[0][1]);
22 }
```

### NOTE

树形 DP 通常使用后序遍历（先处理子节点，再处理父节点），这保证了计算父节点时子节点的状态已经确定。

## 5.3 高级技巧与优化

## I 换根 DP (二次扫描)

换根 DP 用于解决以每个节点为根时的相关问题。基本思路：

1. 第一次 DFS：任选一个根，计算以该根为根的树的相关信息
2. 第二次 DFS：通过换根操作，快速计算以其他节点为根时的信息

应用场景：

- 树的重心
- 树上所有节点到其他节点的距离和
- 树的直径端点

### TIP

换根 DP 的关键在于推导从一个根转移到相邻节点根时的状态变化公式。

## I 虚树优化

当树的节点很多，但只有少数关键点需要处理时，可以构建虚树：

- 只保留关键点和它们的最近公共祖先（LCA）
- 在虚树上进行 DP，大幅减少计算量
- 时间复杂度从  $O(n)$  降低到  $O(k \log k)$ ，其中  $k$  是关键点数量

# 6 状态压缩 DP

状态压缩动态规划是一种高效处理组合优化问题的技术，特别适用于状态空间呈指数级增长的问题。

## 6.1 基本概念

状态压缩 DP 的核心思想：

- 状态压缩：使用整数的二进制位表示状态，每个位对应一个布尔值（0/1）
- 位运算：使用位操作快速进行状态转移和状态检查
- 状态空间优化：将指数级状态空间压缩到有限整数范围内

使用场景：

- 状态维度高但每个维度取值有限（通常为布尔值）
- 状态空间可达  $2^{20}$ （约 100 万）以内
- 问题具有最优子结构和无后效性

核心技巧：

- 状态表示： $dp[state][*]$ ，其中  $state$  是整数表示的压缩状态
- 位运算操作：
  - 检查状态：`state & (1 << i)`
  - 设置状态：`state | (1 << i)`
  - 清除状态：`state & ~(1 << i)`
  - 状态交集：`state1 & state2`
  - 状态并集：`state1 | state2`

## 6.2 经典问题

## I 旅行商问题 (TSP)

旅行商问题：给定  $n$  个城市和城市之间的距离，求从起点出发，访问所有城市恰好一次并回到起点的最短路径。

状态定义： $dp[mask][i]$  表示已访问城市集合为  $mask$ ，当前在城市  $i$  的最短路径长度

状态转移方程： $dp[mask][i] = \min(dp[mask \oplus (1 \ll i)][j] + dist[j][i])$ ，其中  $j$  是  $mask$  中除  $i$  外的城市

初始条件： $dp[1][0] = 0$ （从城市 0 出发）

最终答案： $\min(dp[(1 \ll n) - 1][i] + dist[i][0])$ ，其中  $i \neq 0$

```

1  int tsp(vector<vector<int>>& dist) {
2      int n = dist.size();
3      int fullMask = (1 << n) - 1;
4      vector<vector<int>> dp(fullMask + 1, vector<int>(n, INT_MAX));
5
6      // 初始状态：从城市0出发
7      dp[1][0] = 0;
8
9      // 枚举所有状态
10     for (int mask = 1; mask <= fullMask; ++mask) {
11         for (int i = 0; i < n; ++i) {
12             if (!(mask & (1 << i))) continue; // 城市i不在mask中
13             if (dp[mask][i] == INT_MAX) continue;
14
15             // 尝试前往未访问的城市
16             for (int j = 0; j < n; ++j) {
17                 if (mask & (1 << j)) continue; // 城市j已访问
18                 int newMask = mask | (1 << j);
19                 dp[newMask][j] = min(dp[newMask][j], dp[mask][i] + dist[i][j]);
20             }
21         }
22     }
23
24     // 回到起点
25     int ans = INT_MAX;
26     for (int i = 1; i < n; ++i) {
27         if (dp[fullMask][i] != INT_MAX) {
28             ans = min(ans, dp[fullMask][i] + dist[i][0]);
29         }
30     }
31
32     return ans;
33 }

```

### CAUTION

TSP 问题的时间复杂度为  $O(2^n \cdot n^2)$ ，适用于  $n \leq 20$  的小规模问题。

## I 棋盘覆盖问题 (多米诺骨牌)

用  $1 \times 2$  的多米诺骨牌覆盖  $n \times m$  的棋盘，求方案数。

状态定义:  $dp[i][mask]$  表示处理到第  $i$  行, 当前行的覆盖状态为  $mask$  的方案数

状态转移: 枚举下一行的所有合法状态, 进行转移

**NOTE**

棋盘覆盖问题通常需要预处理所有合法状态和状态之间的转移关系。

## 6.3 优化技巧

### 滚动数组优化

如果状态转移只依赖于前一层的状态, 可以使用滚动数组将空间复杂度从  $O(2^n)$  降低到  $O(2^{n-1})$ 。

### 状态子集枚举优化

对于某些问题, 可以只枚举状态的子集, 而不是所有状态, 从而减少计算量。

子集枚举技巧:

```
1 // 枚举 mask 的所有子集
2 for (int sub = mask; sub; sub = (sub - 1) & mask) {
3     // 处理子集 sub
4 }
```

**TIP**

子集枚举的时间复杂度为  $O(3^n)$ , 比枚举所有状态对的  $O(4^n)$  更优。

## 7 数位 DP

数位动态规划是解决与数字的数位相关问题的强大技术, 特别适用于统计满足特定条件的数字数量、计算数字和等场景。

### 7.1 基本概念

数位 DP 结合了动态规划和深度优先搜索的思想:

- 数位处理: 将数字按位分解 (十进制或二进制)
- 状态压缩: 记录当前处理位置、是否受限、前导零状态等信息
- 记忆化搜索: 存储已计算状态避免重复计算

适用场景:

- 统计区间  $[A, B]$  内满足特定条件的数字数量
- 计算数字的数位属性 (和、乘积、特定模式等)
- 处理超大范围数字 ( $10^{18}$  级别)

关键技巧:

- 状态表示:  $dp[pos][tight][leading\_zero][state]$ 
  - $pos$ : 当前处理的位置 (从高位到低位)
  - $tight$ : 是否受限 (当前位能否任意取值)
  - $leading\_zero$ : 是否存在前导零
  - $state$ : 问题特定状态 (如前几位值、和、乘积等)

- 位运算：高效处理二进制相关问题
- 前导零处理：区分真实 0 和前导 0

## 7.2 经典问题

### 统计不含连续 1 的二进制数

统计  $[0, n]$  范围内，二进制表示中不含连续 1 的数字个数。

状态定义： $dp[pos][prev\_one][tight]$

- $pos$ ：当前处理的位
- $prev\_one$ ：前一位是否为 1
- $tight$ ：是否受限于  $n$  的对应位

状态转移：

- 如果前一位是 1，当前位只能填 0
- 如果前一位是 0，当前位可以填 0 或 1（需考虑  $tight$  限制）

```

1  vector<vector<vector<int>>> memo;
2  string binary;
3
4  int dfs(int pos, bool prevOne, bool tight) {
5      if (pos == binary.size()) return 1;
6      if (!tight && memo[pos][prevOne][0] != -1)
7          return memo[pos][prevOne][0];
8      if (tight && memo[pos][prevOne][1] != -1)
9          return memo[pos][prevOne][1];
10
11     int limit = tight ? (binary[pos] - '0') : 1;
12     int result = 0;
13
14     for (int digit = 0; digit <= limit; ++digit) {
15         if (prevOne && digit == 1) continue; // 不能有连续的1
16         result += dfs(pos + 1, digit == 1, tight && (digit == limit));
17     }
18
19     if (!tight) memo[pos][prevOne][0] = result;
20     else memo[pos][prevOne][1] = result;
21
22     return result;
23 }
24
25 int countNumbers(int n) {
26     // 转换为二进制
27     binary = "";
28     while (n > 0) {
29         binary = char('0' + (n % 2)) + binary;
30         n /= 2;
31     }
32     if (binary.empty()) binary = "0";
33
34     memo.assign(binary.size(), vector<vector<int>>(2, vector<int>(2, -1)));

```

```

35     return dfs(0, false, true);
36 }

```

## 统计区间内数位和等于 K 的数字个数

统计  $[A, B]$  范围内，数位和等于 K 的数字个数。

解题思路：

- 转化为求  $[0, B]$  和  $[0, A - 1]$  的结果之差
- 状态： $dp[pos][sum][tight]$
- 转移：枚举当前位的数字（0-9），累加到 sum

### NOTE

数位 DP 通常使用记忆化搜索实现，需要注意处理前导零和边界条件。

## 8 DP 优化技巧总结

### 8.1 空间优化

#### 滚动数组

当状态转移只依赖于前一层或前几个状态时，可以使用滚动数组减少空间复杂度。

示例：0-1 背包从  $O(nW)$  优化到  $O(W)$

### 8.2 时间优化

#### 单调队列优化

适用于状态转移方程形如： $dp[i] = \frac{\min}{\max(dp[j] + f(i, j))}$  的问题。

核心思想：维护一个单调队列，快速找到最优的 j。

时间复杂度：从  $O(n^2)$  优化到  $O(n)$

#### 斜率优化

适用于状态转移方程可以转化为斜率形式的问题。

核心思想：将状态看作平面上的点，通过维护凸包快速找到最优决策点。

应用场景：

- 任务调度问题
- 仓库建设问题

#### 决策单调性分治优化

如果决策点具有单调性（即  $opt[i] \leq opt[i + 1]$ ），可以使用分治优化。

核心思想：

- 对于区间  $[l, r]$ ，已知最优决策点在  $[optL, optR]$  范围内

- 计算中间位置 `mid` 的最优决策点 `optMid`
- 递归处理 `[l, mid - 1]` 和 `[mid + 1, r]`

时间复杂度：从  $O(n^2)$  优化到  $O(n \log n)$

#### TIP

掌握这些优化技巧需要在大量练习中积累经验，建议先从基础的线性 DP 和背包问题入手，逐步挑战更复杂的优化问题。

## 9 线性 DP 补充

### 9.1 最长公共子序列 (LCS)

给定两个字符串 `text1` 和 `text2`，返回这两个字符串的最长公共子序列的长度。

状态定义： $dp[i][j]$  表示 `text1` 的前  $i$  个字符和 `text2` 的前  $j$  个字符的最长公共子序列长度

状态转移方程：

- 如果 `text1[i - 1] == text2[j - 1]`： $dp[i][j] = dp[i - 1][j - 1] + 1$
- 否则： $dp[i][j] = \max(dp[i - 1][j], dp[i][j - 1])$

初始条件： $dp[0][j] = 0$ ,  $dp[i][0] = 0$

```

1  int longestCommonSubsequence(string text1, string text2) {
2      int m = text1.size(), n = text2.size();
3      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
4
5      for (int i = 1; i <= m; ++i) {
6          for (int j = 1; j <= n; ++j) {
7              if (text1[i - 1] == text2[j - 1]) {
8                  dp[i][j] = dp[i - 1][j - 1] + 1;
9              } else {
10                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
11             }
12         }
13     }
14
15     return dp[m][n];
16 }

```

#### NOTE

LCS 问题是经典的二维 DP 问题，时间复杂度为  $O(mn)$ 。

### 9.2 编辑距离 (Edit Distance)

给定两个单词 `word1` 和 `word2`，计算出将 `word1` 转换成 `word2` 所使用的最少操作数。可以进行的操作有：插入一个字符、删除一个字符、替换一个字符。

状态定义： $dp[i][j]$  表示将 `word1` 的前  $i$  个字符转换为 `word2` 的前  $j$  个字符所需的最少操作数

状态转移方程：

- 如果  $word1[i-1] == word2[j-1]$ :  $dp[i][j] = dp[i-1][j-1]$
- 否则:  $dp[i][j] = \min(dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + 1)$ 
  - $dp[i-1][j] + 1$ : 删除操作
  - $dp[i][j-1] + 1$ : 插入操作
  - $dp[i-1][j-1] + 1$ : 替换操作

初始条件:  $dp[i][0] = i$ ,  $dp[0][j] = j$

```

1  int minDistance(string word1, string word2) {
2      int m = word1.size(), n = word2.size();
3      vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
4
5      // 初始化边界
6      for (int i = 0; i <= m; ++i) dp[i][0] = i;
7      for (int j = 0; j <= n; ++j) dp[0][j] = j;
8
9      for (int i = 1; i <= m; ++i) {
10         for (int j = 1; j <= n; ++j) {
11             if (word1[i - 1] == word2[j - 1]) {
12                 dp[i][j] = dp[i - 1][j - 1];
13             } else {
14                 dp[i][j] = min({dp[i - 1][j] + 1,          // 删除
15                               dp[i][j - 1] + 1,          // 插入
16                               dp[i - 1][j - 1] + 1}); // 替换
17             }
18         }
19     }
20
21     return dp[m][n];
22 }

```

#### TIP

编辑距离是自然语言处理中的重要算法，用于拼写检查、DNA 序列比对等场景。

## 9.3 买股票的最佳时机系列

### 买卖股票的最佳时机 I（只能交易一次）

状态定义：

- $dp[i][0]$ : 第  $i$  天不持有股票的最大利润
- $dp[i][1]$ : 第  $i$  天持有股票的最大利润

状态转移方程：

- $dp[i][0] = \max(dp[i-1][0], dp[i-1][1] + prices[i])$
- $dp[i][1] = \max(dp[i-1][1], -prices[i])$

```

1  int maxProfit(vector<int>& prices) {
2      int n = prices.size();
3      if (n == 0) return 0;
4
5      vector<vector<int>> dp(n, vector<int>(2, 0));

```

```

6    dp[0][0] = 0;
7    dp[0][1] = -prices[0];
8
9    for (int i = 1; i < n; ++i) {
10       dp[i][0] = max(dp[i - 1][0], dp[i - 1][1] + prices[i]);
11       dp[i][1] = max(dp[i - 1][1], -prices[i]);
12    }
13
14    return dp[n - 1][0];
15 }

```

## 买卖股票的最佳时机 II (无限次交易)

状态转移方程：

- $dp[i][0] = \max(dp[i - 1][0], dp[i - 1][1] + prices[i])$
- $dp[i][1] = \max(dp[i - 1][1], dp[i - 1][0] - prices[i])$

### NOTE

与版本 I 的区别在于：买入时可以从之前的利润中扣除，而不是从 0 开始。

## 10 概率 DP 与期望 DP

概率 DP 和期望 DP 是动态规划的重要分支，用于解决涉及随机性的问题。

### 10.1 基本概念

概率 DP：计算达到某种状态的概率

期望 DP：计算达到某种状态的期望值（通常是步数、代价等）

核心公式：

- 期望的线性性： $E[X + Y] = E[X] + E[Y]$
- 全期望公式： $E[X] = \sum(E[X|A_i] \cdot P(A_i))$

### 10.2 经典问题

#### 掷骰子问题

有一个  $n$  面的骰子，问掷出所有面的期望次数是多少？

状态定义： $dp[i]$  表示已经掷出  $i$  个不同面，还需要掷多少次才能掷出所有  $n$  个面的期望

状态转移方程：

$$dp[i] = \frac{i}{n} \cdot dp[i] + \frac{n-i}{n} \cdot dp[i+1] + 1$$

$$\text{化简得: } dp[i] = dp[i+1] + \frac{n-i}{n}$$

初始条件： $dp[n] = 0$

最终答案： $dp[0]$

```
1 double expectedRolls(int n) {
```

 C++

```
2    vector<double> dp(n + 1, 0);
3    dp[n] = 0;
4
5    for (int i = n - 1; i >= 0; --i) {
6        dp[i] = dp[i + 1] + (double)n / (n - i);
7    }
8
9    return dp[0];
10 }
```

**NOTE**

这个问题的答案是调和级数： $H_n = 1 + \frac{1}{2} + \frac{1}{3} + \dots + \frac{1}{n}$

## 收集卡片问题

有  $n$  种卡片，每次随机获得一种，问收集齐所有卡片的期望次数。

这个问题与掷骰子问题本质相同，答案也是  $n \cdot H_n$ 。

## 10.3 解题技巧

**逆向思维：**期望 DP 通常从目标状态逆向推导到初始状态

**状态设计：**

- 概率 DP：状态通常表示“已经达到某种条件”
- 期望 DP：状态通常表示“从当前状态到目标状态的期望步数”

**边界处理：**

- 概率 DP：初始状态概率为 1，其他为 0
- 期望 DP：目标状态的期望为 0

**CAUTION**

期望 DP 容易出现循环依赖，需要仔细设计状态转移顺序，或者使用高斯消元求解方程组。

## 11 DP 建模思路总结

### 11.1 如何判断一个问题是否可以用 DP 解决

#### 关键特征

- **最优子结构：**问题的最优解包含子问题的最优解
- **重叠子问题：**相同的子问题会被多次计算
- **无后效性：**未来的决策只与当前状态有关，与如何到达当前状态无关

#### 识别信号

- 问题要求“最大值”、“最小值”、“方案数”
- 问题涉及“选择”或“不选择”

- 问题可以分解为规模更小的子问题
- 暴力递归存在大量重复计算

## 11.2 DP 问题建模步骤

### 第一步：确定状态

思考方式：

- 问题的变量有哪些？
- 哪些变量会影响最终结果？
- 如何用这些变量唯一标识一个子问题？

常见状态类型：

- 位置/索引： $dp[i]$ 、 $dp[i][j]$
- 集合： $dp[mask]$ （状压 DP）
- 剩余容量： $dp[i][w]$ （背包）
- 前缀/后缀： $dp[i]$  表示前  $i$  个元素

### 第二步：确定状态转移方程

思考方式：

- 当前状态可以由哪些状态转移而来？
- 每种转移的代价/收益是什么？
- 如何选择最优的转移？

常见转移类型：

- 线性转移： $dp[i]$  从  $dp[i-1]$ 、 $dp[i-2]$  等转移
- 区间转移： $dp[i][j]$  从  $dp[i][k]$  和  $dp[k+1][j]$  转移
- 集合转移： $dp[mask]$  从  $dp[mask \oplus (1 \ll i)]$  转移

### 第三步：确定初始条件

思考方式：

- 最小子问题的解是什么？
- 边界情况如何处理？

常见初始条件：

- $dp[0] = 0$  或  $dp[0] = 1$ （取决于问题）
- $dp[i][i] = 0$ （区间 DP）
- $dp[0][w] = 0$ （背包问题）

### 第四步：确定计算顺序

思考方式：

- 计算  $dp[i]$  时，它依赖的状态是否已经计算过？
- 应该从小到大还是从大到小枚举？

常见顺序：

- 线性 DP：从小到大枚举  $i$
- 区间 DP：按区间长度从小到大

- 树形 DP: 后序遍历 (先子节点后父节点)
- 状压 DP: 按 mask 从小到大

11.3 常见错误与调试技巧

常见错误

- 状态定义不清: 导致转移方程难以推导
- 边界条件错误: 特别是  $dp[0]$  的值
- 转移顺序错误: 依赖的状态还未计算
- 数组越界: 访问  $dp[-1]$  或  $dp[n]$
- 初始化遗漏: 某些状态未正确初始化

调试技巧

- 打印 DP 表: 观察中间状态是否符合预期
- 小规模测试: 用手工计算验证小数据的结果
- 对比暴力解: 用小数据对比 DP 和暴力解的结果
- 检查边界: 特别关注  $i = 0$ 、 $i = n - 1$  等边界情况

TIP

DP 问题的难点不在于代码实现, 而在于正确的状态定义和转移方程设计。多做题、多总结是提高 DP 能力的关键。

12 DP 问题分类速查

12.1 按问题类型分类

| 类型    | 典型问题         | 状态维度        |
|-------|--------------|-------------|
| 线性 DP | LIS、LCS、编辑距离 | 1D 或 2D     |
| 背包 DP | 0-1 背包、完全背包  | 2D          |
| 区间 DP | 石子合并、矩阵连乘    | 2D          |
| 树形 DP | 树的直径、最大独立集   | 树上 DFS      |
| 状压 DP | TSP、棋盘覆盖     | mask + 其他   |
| 数位 DP | 数字计数、数位统计    | pos + tight |
| 概率 DP | 期望步数、概率计算    | 视问题而定       |

12.2 按优化技巧分类

| 优化方法 | 适用场景   | 复杂度改进                      |
|------|--------|----------------------------|
| 滚动数组 | 只依赖前一层 | $O(n) \rightarrow O(1)$ 空间 |
| 单调队列 | 滑动窗口最值 | $O(n^2) \rightarrow O(n)$  |

---

|        |         |                                  |
|--------|---------|----------------------------------|
| 斜率优化   | 特定形式的转移 | $O(n^2) \rightarrow O(n)$        |
| 四边形不等式 | 区间 DP   | $O(n^3) \rightarrow O(n^2)$      |
| 决策单调性  | 决策点单调   | $O(n^2) \rightarrow O(n \log n)$ |

---



本章完

# Chapter 18

## 回溯法与分支限界

---

# Chapter 19

## 搜索优化与高级技巧

---

# Chapter 20

## 其他算法技巧

---



# 高级数据结构

|                     |    |
|---------------------|----|
| 21. 线段树与树状数组 ·····  | 51 |
| 22. 平衡树进阶 ·····     | 52 |
| 23. 分块与莫队算法 ·····   | 53 |
| 24. 并查集与高级应用 ·····  | 54 |
| 25. 字符串高级数据结构 ····· | 55 |
| 26. 图论高级算法 ·····    | 56 |
| 27. 网络流与匹配 ·····    | 57 |
| 28. 其他高级数据结构 ·····  | 58 |

# Chapter 21

## 线段树与树状数组

---

# Chapter 22

## 平衡树进阶

---

# Chapter 23

## 分块与莫队算法

---

# Chapter 24

## 并查集与高级应用

---

# Chapter 25

## 字符串高级数据结构

---

# Chapter 26

## 图论高级算法

---

# Chapter 27

## 网络流与匹配

---

# Chapter 28

## 其他高级数据结构

---